

PROJECT
On
Predictive Modeling and Statistical Decomposition of Urban Traffic
Collision Severity Using Multi-Source Temporal Crash Data
In

DPEL:Data Preprocessing and EDA Lab

BACHELOR OF TECHNOLOGY
IN

Artificial Intelligence and Machine Learning

SUBMITTED BY
Name:Rami Baggash
PRN:25070126138

Name:Sakher Shabil
PRN:25070126150

1. Introduction

Road traffic collisions are a major public safety issue, particularly in urban environments where traffic density, infrastructure complexity, and heterogeneous driving behavior increase both the frequency and severity of accidents. While many collisions result in minor damage, a significant proportion lead to serious injuries or fatalities, making the analysis of collision severity a critical problem in transportation safety.

Collision severity is influenced by a combination of environmental conditions, vehicle characteristics, and human factors. However, traditional studies often analyze these factors independently, failing to capture their joint effects and interactions. In addition, collision patterns evolve over time, requiring a temporal perspective to fully understand underlying trends.

This study addresses these limitations by leveraging the UK Department for Transport STATS19 dataset, integrating three interrelated data sources: collisions, vehicles, and casualties. The analysis is conducted over a multi-year period, enabling both cross-sectional and temporal examination of traffic accidents.

The objective of this work is twofold. First, to develop predictive models for estimating collision severity using multi-source structured data. Second, to perform statistical decomposition of contributing factors, identifying the relative importance and interaction effects of variables influencing severity outcomes.

Given that the target variable, *collision_severity*, is ordinal (Fatal, Serious, Slight), the modeling approach accounts for the ordered nature of the outcome. Temporal features derived from date and time variables are also incorporated to capture seasonal and time-dependent patterns.

The findings of this study aim to support data-driven decision-making in traffic management, enhance risk assessment strategies, and contribute to the development of safer urban transportation systems..

A). Data Understanding

2 Problem Statement (Real-World Relevance)

The objective of this study is to model and predict road traffic collision severity using multi-source structured data. The problem is formulated as an ordinal classification task, where the target variable *collision_severity* takes ordered values: Fatal (1), Serious (2), and Slight (3).

The input data is derived from three interconnected datasets—Collisions, Vehicles, and Casualties—which are linked using the common key *collision_index*. Since the prediction is performed at the collision level, vehicle-level and casualty-level data are aggregated to generate meaningful features for each collision.

The resulting feature space includes:

- 1)Environmental and road attributes (from the Collisions dataset)
- 2)Aggregated vehicle characteristics (e.g., number of vehicles, average driver age)
- 3)Aggregated casualty information (e.g., number of casualties, demographic summaries)

This multi-source integration enables a comprehensive representation of each collision event.

The modeling task must address several challenges, including:

- 2)Class imbalance, where slight collisions dominate the dataset
- 3)Preservation of ordinal relationships between severity levels
- 4)Integration of heterogeneous feature types (categorical, numerical, temporal)

Model performance will be evaluated using metrics suitable for imbalanced ordinal classification, including weighted F1-score and confusion matrix analysis, ensuring reliable assessment across all severity classes.

The ultimate goal is to identify key determinants of collision severity and improve predictive accuracy to support data-driven road safety interventions.

3. Dataset Description

Data Source

The dataset used in this study is the **UK Department for Transport STATS19 Road Safety dataset**, which contains detailed records of reported road traffic collisions in the United Kingdom. The analysis focuses on data from the **most recent five-year period**, enabling the study of current trends and temporal patterns.

The dataset is organized into three interrelated tables:

- **Collisions**
- **Vehicles**
- **Casualties**

These tables are linked through a common identifier, enabling a unified analysis of accident-level, vehicle-level, and individual-level information.

3.1 Dataset Relationships

The three datasets are connected using the key:

- **collision_index** (unique identifier for each collision)

Relationship structure:

- The **Collisions dataset** contains **one record per collision** (primary entity)
- The **Vehicles dataset** contains **multiple records per collision**
- The **Casualties dataset** contains **multiple records per collision**

This establishes a **one-to-many relationship**, where vehicle and casualty data must be aggregated and merged with the collisions dataset to perform analysis at the collision level.

3.2 Attributes / Features

A) Collisions Dataset (Accident-Level Information)

This dataset provides contextual and environmental details for each collision.

Key attributes include:

- **collision_index** – unique identifier

- `collision_year` – year of occurrence
- `longitude, latitude` – geographical location
- `police_force` – reporting authority
- `number_of_vehicles` – vehicles involved
- `urban_or_rural_area` – type of location
- `trunk_road_flag` – road classification

Additional important variables:

- `road_type`
- `weather_conditions`
- `light_conditions`
- `speed_limit`

These variables are critical for understanding environmental and infrastructural influences on collision severity.

B) Vehicles Dataset (Vehicle-Level Information)

This dataset contains information about each vehicle involved in a collision.

Key attributes include:

- `collision_index` – linking key
- `vehicle_reference` – unique vehicle identifier

- `vehicle_type` – type of vehicle
- `vehicle_manoeuvre` – movement at time of collision
- `vehicle_direction_from`, `vehicle_direction_to` – direction of travel

Driver and vehicle characteristics:

- `age_of_driver`
- `age_band_of_driver`
- `engine_capacity_cc`
- `propulsion_code`
- `age_of_vehicle`
- `generic_make_model`

Socio-demographic indicators:

- `driver_imd_decile`
- `lsoa_of_driver`

C) Casualties Dataset (Individual-Level Information)

This dataset contains information about individuals affected in collisions.

Key attributes include:

- `collision_index` – linking key

- `vehicle_reference` – associated vehicle
- `casualty_reference` – unique identifier
- `casualty_class` – role (driver, passenger, pedestrian)
- `sex_of_casualty`
- `age_of_casualty`
- `age_band_of_casualty`

Severity-related attributes:

- `Casualty_severity`

Note: The variable `casualty_severity` is excluded during modeling to prevent data leakage, as it is strongly correlated with the target variable `collision_severity`.

Additional attributes:

- `casualty_type`
- `casualty_imd_decile`
- `lsoa_of_casualty`

3.3 Dataset Size

The dataset consists of the following number of records and attributes:

[5]	 casualties.shape
✓ 0s	...
▼	(640522, 23)

[6]	 collisions.shape
✓ 0s	...
▼	(503475, 44)

[7]	vehicles.shape
✓ 0s	
▼	(920692, 32)

- Collisions dataset: **503,475 rows and 44 columns**
- Vehicles dataset: **920,692 rows and 32 columns**
- Casualties dataset: **640,522 rows and 23 columns**

3.4 Target Variable

The target variable for this study is:

👉 collision_severity

Value	Severity
1	Fatal
2	Serious
3	Slight

This variable represents the severity of each collision and is treated as an **ordinal variable**, where the order of categories is meaningful.

Additionally, the dataset is expected to exhibit **class imbalance**, with a higher proportion of slight collisions compared to serious and fatal cases. This imbalance must be considered during model development.

4. Initial Observations

4.1 Data Quality Issues Identified

The datasets were examined using structural inspection methods to understand their composition, data types, and overall data quality.

```
[8]
✓ 1s casualties.info()

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 640522 entries, 0 to 640521
Data columns (total 23 columns):
 #   Column                                  Non-Null Count  Dtype
---  -
 0   collision_index                        640522 non-null object
 1   collision_year                        640522 non-null int64
 2   collision_ref_no                      640522 non-null object
 3   vehicle_reference                    640522 non-null int64
 4   casualty_reference                   640522 non-null int64
 5   casualty_class                       640522 non-null int64
 6   sex_of_casualty                     640522 non-null int64
 7   age_of_casualty                     640522 non-null int64
 8   age_band_of_casualty                640522 non-null int64
 9   casualty_severity                   640522 non-null int64
10   pedestrian_location                 640522 non-null int64
11   pedestrian_movement                 640522 non-null int64
12   car_passenger                      640522 non-null int64
13   bus_or_coach_passenger              640522 non-null int64
14   pedestrian_road_maintenance_worker 640522 non-null int64
15   casualty_type                      640522 non-null int64
16   casualty_imd_decile                 640522 non-null int64
17   lsoa_of_casualty                   640522 non-null object
18   enhanced_casualty_severity          640522 non-null int64
19   casualty_injury_based               640522 non-null int64
20   casualty_adjusted_severity_serious  640522 non-null float64
21   casualty_adjusted_severity_slight   640522 non-null float64
22   casualty_distance_banding           640522 non-null int64
dtypes: float64(2), int64(18), object(3)
memory usage: 112.4+ MB
```

```
collisions.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 503475 entries, 0 to 503474
Data columns (total 44 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   collision_index                       503475 non-null object
1   collision_year                       503475 non-null int64
2   collision_ref_no                     503475 non-null object
3   location_easting_osgr               503410 non-null float64
4   location_northing_osgr             503410 non-null float64
5   longitude                           503410 non-null float64
6   latitude                           503410 non-null float64
7   police_force                       503475 non-null int64
8   collision_severity                   503475 non-null int64
9   number_of_vehicles                   503475 non-null int64
10  number_of_casualties                503475 non-null int64
11  date                               503475 non-null object
12  day_of_week                        503475 non-null int64
13  time                               503475 non-null object
14  local_authority_district            503475 non-null int64
15  local_authority_ons_district        503475 non-null object
16  local_authority_highway             503475 non-null object
17  local_authority_highway_current     503395 non-null object
18  first_road_class                    503475 non-null int64
19  first_road_number                   503475 non-null int64
20  road_type                           503475 non-null int64
21  speed_limit                         503475 non-null int64
22  junction_detail_historic            503475 non-null int64
23  junction_detail                     503475 non-null int64
24  junction_control                     503475 non-null int64
25  second_road_class                   503475 non-null int64
26  second_road_number                  503475 non-null int64
27  pedestrian_crossing_human_control_historic 503475 non-null int64
28  pedestrian_crossing_physical_facilities_historic 503475 non-null int64
29  pedestrian_crossing                 503475 non-null int64
30  light_conditions                    503475 non-null int64
31  weather_conditions                 503475 non-null int64
32  road_surface_conditions             503475 non-null int64
33  special_conditions_at_site          503475 non-null int64
34  carriageway_hazards_historic        503475 non-null int64
35  carriageway_hazards                 503475 non-null int64
36  urban_or_rural_area                 503475 non-null int64
37  did_police_officer_attend_scene_of_accident 503475 non-null int64
38  trunk_road_flag                     503475 non-null int64
39  lsoa_of_accident_location           503475 non-null object
40  enhanced_severity_collision         503475 non-null object
41  collision_injury_based               503475 non-null int64
42  collision_adjusted_severity_serious  503475 non-null float64
43  collision_adjusted_severity_slight   503475 non-null float64
dtypes: float64(6), int64(30), object(8)
memory usage: 169.0+ MB
```

```
vehicles.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 920692 entries, 0 to 920691
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   collision_index                       920692 non-null object
1   collision_year                       920692 non-null int64
2   collision_ref_no                     920692 non-null object
3   vehicle_reference                    920692 non-null int64
4   vehicle_type                         920692 non-null int64
5   towing_and_articulation              920692 non-null int64
6   vehicle_manoeuvre_historic           920692 non-null int64
7   vehicle_manoeuvre                   920692 non-null int64
8   vehicle_direction_from               920692 non-null int64
9   vehicle_direction_to                 920692 non-null int64
10  vehicle_location_restricted_lane_historic 920692 non-null int64
11  vehicle_location_restricted_lane     920692 non-null int64
12  junction_location                    920692 non-null int64
13  skidding_and_overturning             920692 non-null int64
14  hit_object_in_carriageway            920692 non-null int64
15  vehicle_leaving_carriageway          920692 non-null int64
16  hit_object_off_carriageway           920692 non-null int64
17  first_point_of_impact                920692 non-null int64
18  vehicle_left_hand_drive              920692 non-null int64
19  journey_purpose_of_driver_historic     920692 non-null int64
20  journey_purpose_of_driver              920692 non-null int64
21  sex_of_driver                        920692 non-null int64
22  age_of_driver                        920692 non-null int64
23  age_band_of_driver                   920692 non-null int64
24  engine_capacity_cc                   920692 non-null int64
25  propulsion_code                      920692 non-null int64
26  age_of_vehicle                       920692 non-null int64
27  generic_make_model                   920692 non-null object
28  driver_imd_decile                    920692 non-null int64
29  lsoa_of_driver                       920692 non-null object
30  scooter_flag                         920692 non-null int64
31  driver_distance_banding              920692 non-null int64
dtypes: int64(28), object(4)
memory usage: 224.8+ MB
```

The analysis shows that all three datasets—Collisions, Vehicles, and Casualties—contain a mix of numerical and categorical variables, represented

using data types such as **int64**, **float64**, and **object**. The presence of object-type variables (e.g., `collision_index`, `lsoa_of_casualty`, `generic_make_model`) indicates categorical or identifier-based data that may require encoding or transformation in later stages.

The Collisions dataset contains several columns with **missing values**, particularly in location-related attributes such as `location_easting_osgr`, `location_northing_osgr`, `longitude`, and `latitude`, where the number of non-null entries is lower than the total number of records. This suggests incomplete geographical information for certain accident records.

In contrast, the Vehicles and Casualties datasets appear to have **no explicit missing values**, as all columns contain full non-null counts. However, this does not necessarily imply complete data quality, as missing information may be encoded using placeholder values.

Across all datasets, the presence of **coded values such as -1** is observed in several variables, indicating unknown or undefined data. These coded missing values must be handled appropriately during preprocessing.

Additionally, some variables such as date and time are stored as **object data types**, which may require conversion to appropriate datetime formats for temporal analysis.

Overall, while the datasets are structurally well-defined, they contain a combination of missing values, coded placeholders, and mixed data types that require careful preprocessing before analysis.

4.2 Missing Values / Duplicates / Incorrect Entries

A detailed analysis was conducted to identify missing values, duplicate records, and incorrect or coded entries within the datasets.

[14]  collisions.isnull().sum()[collisions.isnull().sum() > 0]

▼ ... 0

location_easting_osgr	65
location_northing_osgr	65
longitude	65
latitude	65
local_authority_highway_current	80

dtype: int64

[16]  vehicles.isnull().sum()[vehicles.isnull().sum() > 0]

▼ ... 0

dtype: int64

[17] casualties.isnull().sum()[casualties.isnull().sum() > 0]

▼ 0

dtype: int64

[18] collisions.duplicated().sum()

▼ np.int64(0)

[19] vehicles.duplicated().sum()

▼ np.int64(0)

[20]  casualties.duplicated().sum()

▼ ... np.int64(0)

```
[25] (collisions == -1).sum().sum()
✓ Os np.int64(1445559)

[26] (vehicles == -1).sum().sum()
✓ Os np.int64(2391201)

[27] (casualties == -1).sum().sum()
✓ Os ... np.int64(923484)
```

The analysis of missing values shows that the **Collisions dataset contains a small number of missing entries**, specifically in location-related variables such as `location_easting_osgr`, `location_northing_osgr`, `longitude`, and `latitude` (each with 65 missing values), as well as `local_authority_highway_current` (80 missing values). These missing values indicate incomplete geographical and administrative information for certain records.

In contrast, the **Vehicles and Casualties datasets do not contain any explicit missing values**, as all columns have complete non-null entries.

The duplicate check confirms that **no duplicate records are present** in any of the datasets, ensuring data consistency and integrity.

However, further inspection reveals a significant presence of **coded missing values represented by -1** across all datasets. The total number of such entries is notably high, with **1,445,559 occurrences in the Collisions dataset**, **2,391,201 in the Vehicles dataset**, and **923,484 in the Casualties dataset**. These values indicate unknown or unavailable information and must be treated as missing data during preprocessing.

Overall, while explicit missing values are limited, the presence of a large number of coded missing values highlights the need for careful data cleaning and preprocessing before analysis.

B) Data Preprocessing

5. Handling Missing Values

5.1 Identification of Missing Data

Before performing missing value analysis, all coded missing values represented by **-1** were first converted into null values (**NaN**) across all datasets (Collisions, Vehicles, and Casualties). This ensures that missing data is consistently represented and prevents distortion in statistical calculations during imputation.

```
[76] collisions.isnull().sum()[collisions.isnull().sum() > 0]
```

...	0
location_easting_osgr	65
location_northing_osgr	65
longitude	65
latitude	65
local_authority_district	413073
local_authority_highway_current	80
speed_limit	15
junction_detail_historic	59607
junction_detail	14791
junction_control	211708
second_road_class	11477
second_road_number	191038
pedestrian_crossing_human_control_historic	63180
pedestrian_crossing_physical_facilities_historic	63144
pedestrian_crossing	4843
light_conditions	19
weather_conditions	13
road_surface_conditions	3209
special_conditions_at_site	63730
carriageway_hazards_historic	63723
carriageway_hazards	4741
urban_or_rural_area	8
did_police_officer_attend_scene_of_accident	1
trunk_road_flag	35920
enhanced_severity_collision	241319

dtype: int64

[77] ✓ 1s vehicles.isnull().sum()[vehicles.isnull().sum() > 0]

...	0
vehicle_type	6545
towing_and_articulation	7500
vehicle_manoeuvre_historic	117314
vehicle_manoeuvre	12335
vehicle_direction_from	18680
vehicle_direction_to	18810
vehicle_location_restricted_lane_historic	115031
vehicle_location_restricted_lane	10637
junction_location	4196
skidding_and_overturning	10093
hit_object_in_carriageway	10022
vehicle_leaving_carriageway	10117
hit_object_off_carriageway	1818
first_point_of_impact	11770
vehicle_left_hand_drive	940
journey_purpose_of_driver_historic	109914
journey_purpose_of_driver	1827
sex_of_driver	3551
age_of_driver	137322
age_band_of_driver	137322
engine_capacity_cc	233745
propulsion_code	222200
age_of_vehicle	222433
driver_imd_decile	189385
driver_distance_banding	777694

dtype: int64

[78] ✓ 0s casualties.isnull().sum()[casualties.isnull().sum() > 0]

...	0
sex_of_casualty	5740
age_of_casualty	13846
age_band_of_casualty	13846
pedestrian_location	14
pedestrian_movement	13
car_passenger	2737
bus_or_coach_passenger	255
pedestrian_road_maintenance_worker	12077
casualty_type	4935
casualty_imd_decile	69483
enhanced_casualty_severity	271013
casualty_distance_banding	529525

dtype: int64

Figure : Missing Values Before Preprocessing

The datasets were analyzed to identify the presence and nature of missing values. Two types of missing data were observed: **explicit missing values (NaN)** and **coded missing values represented by -1**.

Explicit missing values were primarily found in the Collisions dataset, particularly in location-related attributes such as `location_easting_osgr`, `location_northing_osgr`, `longitude`, and `latitude`, as well as in certain administrative variables. In contrast, the Vehicles and Casualties datasets initially appeared to have no missing values; however, further inspection revealed a large number of coded missing values.

The value `-1` was used across multiple variables to indicate unknown or unavailable information. To ensure consistency in data preprocessing, all occurrences of `-1` were converted to null values (NaN), enabling uniform handling of missing data across all datasets.

5.2 Missing Value Treatment and Justification

```
[70] ✓ Os collisions.replace(-1, np.nan, inplace=True)
```

```
[71] ✓ Os vehicles.replace(-1, np.nan, inplace=True)
```

```
[72] ✓ Os casualties.replace(-1, np.nan, inplace=True)
```

To handle missing values effectively, different imputation strategies were applied based on the type of variable and the proportion of missing data.

For numerical variables with a relatively small proportion of missing values, such as location-related attributes (`longitude`, `latitude`, `location_easting_osgr`, `location_northing_osgr`) and age-related variables (`age_of_driver`, `age_of_casualty`), median imputation was applied.

The median was chosen because these variables exhibit skewed distributions and may contain outliers. Using the median reduces the influence of extreme values and provides a more robust estimate of central tendency compared to the mean.

For categorical variables, including attributes such as `vehicle_type`, `weather_conditions`, `light_conditions`, `casualty_type`, and other

vehicle and road-related features, missing values were handled using **mode imputation**. The mode preserves the most frequent category and maintains the original distribution of categorical data.

Variables with a **moderate level of missing data** were selectively imputed where appropriate, ensuring that meaningful information was retained without introducing excessive bias.

However, variables with a **high proportion of missing values**, such as `driver_distance_banding`, `enhanced_severity_collision`, `casualty_distance_banding`, and several historical attributes, were **not imputed**. Imputing these variables would introduce unreliable values and potentially distort the analysis. These features will be handled separately during the data cleaning stage.

Additionally, certain derived variables, such as `age_band_of_casualty`, were not imputed, as they are redundant representations of already available continuous variables (`age_of_casualty`) and do not provide additional analytical value. Overall, this approach ensures that missing values are handled systematically while preserving data integrity, minimizing bias, and maintaining the reliability of subsequent analysis.

This variable-specific imputation strategy ensures that missing values are handled in a manner consistent with the underlying data distribution, minimizing bias and preserving the integrity of the dataset for subsequent analysis.

Handling missing values of Collision:

```
#Small Missing
# Numerical -> MEDIAN
collisions['location_easting_osgr'].fillna(collisions['location_easting_osgr'].median(), inplace=True)
collisions['location_northing_osgr'].fillna(collisions['location_northing_osgr'].median(), inplace=True)
collisions['longitude'].fillna(collisions['longitude'].median(), inplace=True)
collisions['latitude'].fillna(collisions['latitude'].median(), inplace=True)
collisions['speed_limit'].fillna(collisions['speed_limit'].median(), inplace=True)

# Categorical -> MODE
collisions['light_conditions'].fillna(collisions['light_conditions'].mode()[0], inplace=True)
collisions['weather_conditions'].fillna(collisions['weather_conditions'].mode()[0], inplace=True)
collisions['urban_or_rural_area'].fillna(collisions['urban_or_rural_area'].mode()[0], inplace=True)
collisions['did_police_officer_attend_scene_of_accident'].fillna(collisions['did_police_officer_attend_scene_of_accident'].mode()[0], inplace=True)
collisions['local_authority_highway_current'].fillna(collisions['local_authority_highway_current'].mode()[0], inplace=True)
collisions['carriageway_hazards'].fillna(collisions['carriageway_hazards'].mode()[0], inplace=True)
collisions['special_conditions_at_site'] = collisions['special_conditions_at_site'].fillna(collisions['special_conditions_at_site'].mode()[0])
collisions['trunk_road_flag'] = collisions['trunk_road_flag'].fillna(collisions['trunk_road_flag'].mode()[0])

#Medium Missing
collisions['junction_detail'].fillna(collisions['junction_detail'].mode()[0], inplace=True)
collisions['second_road_class'].fillna(collisions['second_road_class'].mode()[0], inplace=True)
collisions['road_surface_conditions'].fillna(collisions['road_surface_conditions'].mode()[0], inplace=True)
collisions['pedestrian_crossing'].fillna(collisions['pedestrian_crossing'].mode()[0], inplace=True)
collisions['junction_control'] = collisions['junction_control'].fillna('Unknown')
```

```
[81]
✓ Os collisions.isnull().sum()[collisions.isnull().sum() > 0]

... 0

local_authority_district 413073
junction_detail_historic 59607
junction_control 211708
second_road_number 191038
pedestrian_crossing_human_control_historic 63180
pedestrian_crossing_physical_facilities_historic 63144
special_conditions_at_site 63730
carriageway_hazards_historic 63723
trunk_road_flag 35920
enhanced_severity_collision 241319

dtype: int64
```

Handling missing values of Vechiles:

```
[82]
✓ Os #Small Missing
vehicles['vehicle_type'].fillna(vehicles['vehicle_type'].mode()[0], inplace=True)
vehicles['towing_and_articulation'].fillna(vehicles['towing_and_articulation'].mode()[0], inplace=True)
vehicles['vehicle_manoeuvre'].fillna(vehicles['vehicle_manoeuvre'].mode()[0], inplace=True)
vehicles['junction_location'].fillna(vehicles['junction_location'].mode()[0], inplace=True)
vehicles['sex_of_driver'].fillna(vehicles['sex_of_driver'].mode()[0], inplace=True)
vehicles['journey_purpose_of_driver'].fillna(vehicles['journey_purpose_of_driver'].mode()[0], inplace=True)
vehicles['first_point_of_impact'].fillna(vehicles['first_point_of_impact'].mode()[0], inplace=True)
vehicles['hit_object_off_carriageway'].fillna(vehicles['hit_object_off_carriageway'].mode()[0], inplace=True)
vehicles['vehicle_left_hand_drive'].fillna(vehicles['vehicle_left_hand_drive'].mode()[0], inplace=True)
```

```
[83]
✓ Os #Medium Missing
vehicles['age_of_driver'].fillna(vehicles['age_of_driver'].median(), inplace=True)
vehicles['engine_capacity_cc'].fillna(vehicles['engine_capacity_cc'].median(), inplace=True)
vehicles['vehicle_direction_from'].fillna(vehicles['vehicle_direction_from'].mode()[0], inplace=True)
vehicles['vehicle_direction_to'].fillna(vehicles['vehicle_direction_to'].mode()[0], inplace=True)
vehicles['vehicle_location_restricted_lane'].fillna(vehicles['vehicle_location_restricted_lane'].mode()[0], inplace=True)
vehicles['skidding_and_overturning'].fillna(vehicles['skidding_and_overturning'].mode()[0], inplace=True)
vehicles['hit_object_in_carriageway'].fillna(vehicles['hit_object_in_carriageway'].mode()[0], inplace=True)
vehicles['vehicle_leaving_carriageway'].fillna(vehicles['vehicle_leaving_carriageway'].mode()[0], inplace=True)
```

```
[84]
✓ Os vehicles.isnull().sum()[vehicles.isnull().sum() > 0]

... 0

vehicle_manoeuvre_historic 117314
vehicle_location_restricted_lane_historic 115031
journey_purpose_of_driver_historic 109914
age_band_of_driver 137322
propulsion_code 222200
age_of_vehicle 222433
driver_jmd_decile 189385
driver_distance_banding 777694

dtype: int64
```

Handling missing values of Casualties:

```
[85]
✓ Os
##Small Missing
casualties['sex_of_casualty'].fillna(casualties['sex_of_casualty'].mode()[0], inplace=True)
casualties['casualty_type'].fillna(casualties['casualty_type'].mode()[0], inplace=True)
casualties['car_passenger'].fillna(casualties['car_passenger'].mode()[0], inplace=True)
casualties['bus_or_coach_passenger'].fillna(casualties['bus_or_coach_passenger'].mode()[0], inplace=True)
casualties['pedestrian_road_maintenance_worker'].fillna(casualties['pedestrian_road_maintenance_worker'].mode()[0], inplace=True)
casualties['pedestrian_location'].fillna(casualties['pedestrian_location'].mode()[0], inplace=True)
casualties['pedestrian_movement'].fillna(casualties['pedestrian_movement'].mode()[0], inplace=True)

[86]
▶ ##Medium Missing
casualties['age_of_casualty'].fillna(casualties['age_of_casualty'].median(), inplace=True)

[87]
▶ casualties.isnull().sum()[casualties.isnull().sum() > 0]
```

	0
age_band_of_casualty	13846
casualty_imd_decile	69483
enhanced_casualty_severity	271013
casualty_distance_banding	529525

dtype: int64

6. Data Cleaning

6.1 Removal of Duplicate Records

The datasets were examined for duplicate entries to ensure data consistency and integrity. A duplicate check was performed on all three datasets (Collisions, Vehicles, and Casualties).

The results indicated that **no duplicate records were present** in any of the datasets. Therefore, no duplicate removal was required at this stage.

```
[ ]
collisions.duplicated().sum()
vehicles.duplicated().sum()
casualties.duplicated().sum()

np.int64(0)
```

6.2 Correction of Inconsistent and Invalid Values

During the preprocessing stage, certain inconsistencies were identified in the dataset. As described in the missing value handling stage, coded values

represented by -1 were previously converted to NaN to ensure consistent treatment of missing data across all datasets.

In addition, variables with **high proportions of missing data** or limited analytical value were removed to improve data quality and reduce noise. These included:

- Historical variables (e.g., *_historic features)
- High-missing attributes (e.g., distance banding variables)
- Redundant or derived variables (e.g., age band variables)

This process ensured that only relevant and reliable features were retained for further analysis.

```
collisions.replace(-1, np.nan, inplace=True)
```

```
vehicles.replace(-1, np.nan, inplace=True)
```

```
casualties.replace(-1, np.nan, inplace=True)
```

6.3 Removal of Irrelevant and Redundant Features

To improve data quality and reduce noise, irrelevant and redundant features were removed from the dataset.

Historical attributes (e.g., variables ending with **_historic**) were excluded as they represent outdated or duplicate information.

Columns with a high proportion of missing values were removed to prevent unreliable imputations and avoid introducing bias into the model.

Additionally, non-informative identifiers (such as unique IDs and reference numbers) and derived or banded variables were excluded, as they do not contribute meaningful predictive value.

This process ensures that the dataset remains focused, consistent, and suitable for robust machine learning modeling.

```

collisions.drop(columns=[
    'local_authority_district',
    'enhanced_severity_collision',
    'second_road_number'
], inplace=True)

vehicles.drop(columns=[
    'age_of_vehicle',
    'propulsion_code',
    'driver_imd_decile'
], inplace=True)

casualties.drop(columns=['casualty_imd_decile'], inplace=True)

collisions.drop(columns=[
    'junction_detail_historic',
    'pedestrian_crossing_human_control_historic',
    'pedestrian_crossing_physical_facilities_historic',
    'carriageway_hazards_historic'
], inplace=True, errors='ignore')

vehicles.drop(columns=[
    'vehicle_manoeuvre_historic',
    'vehicle_location_restricted_lane_historic',
    'journey_purpose_of_driver_historic',
    'driver_distance_banding',
    'age_band_of_driver'
], inplace=True, errors='ignore')

casualties.drop(columns=[
    'casualty_distance_banding',
    'enhanced_casualty_severity',
    'age_band_of_casualty'
], inplace=True, errors='ignore')

```

6.3 Data Type Correction

To ensure consistency and suitability for analysis, data types of several variables were corrected.

The **date** and **time** variables were converted to **datetime format** to enable temporal analysis.

Categorical variables that were originally stored as numerical codes (e.g., weather conditions, road type, vehicle type, casualty class) were converted to the **category data type**. This improves memory efficiency and ensures correct interpretation during analysis and modeling.

Numerical variables such as age, geographical coordinates, and counts were retained in their appropriate numeric formats.

```

collisions['date'] = pd.to_datetime(collisions['date'])
collisions['time'] = pd.to_datetime(collisions['time'], errors='coerce')

... /tmp/ipykernel_5367/3516349168.py:1: UserWarning: Parsing dates in %d/%m/%Y format when dayfirst=False (the default) was specified. Pass 'dayfirst=True' or specify a format to silence this warning.
collisions['date'] = pd.to_datetime(collisions['date'])
/tmp/ipykernel_5367/3516349168.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling back to 'dateutil'. To ensure parsing is consistent and as-expected, please specify a format.
collisions['time'] = pd.to_datetime(collisions['time'], errors='coerce')

collisions['junction_detail'] = collisions['junction_detail'].astype('category')
collisions['junction_control'] = collisions['junction_control'].astype('category')
collisions['pedestrian_crossing'] = collisions['pedestrian_crossing'].astype('category')
collisions['road_surface_conditions'] = collisions['road_surface_conditions'].astype('category')
collisions['special_conditions_at_site'] = collisions['special_conditions_at_site'].astype('category')
collisions['carriageway_hazards'] = collisions['carriageway_hazards'].astype('category')
collisions['did_police_officer_attend_scene_of_accident'] = collisions['did_police_officer_attend_scene_of_accident'].astype('category')

vehicles['vehicle_manoeuvre'] = vehicles['vehicle_manoeuvre'].astype('category')
vehicles['vehicle_direction_from'] = vehicles['vehicle_direction_from'].astype('category')
vehicles['vehicle_direction_to'] = vehicles['vehicle_direction_to'].astype('category')
vehicles['skidding_and_overturning'] = vehicles['skidding_and_overturning'].astype('category')
vehicles['hit_object_in_carriageway'] = vehicles['hit_object_in_carriageway'].astype('category')
vehicles['vehicle_leaving_carriageway'] = vehicles['vehicle_leaving_carriageway'].astype('category')
vehicles['hit_object_off_carriageway'] = vehicles['hit_object_off_carriageway'].astype('category')
vehicles['first_point_of_impact'] = vehicles['first_point_of_impact'].astype('category')
vehicles['vehicle_left_hand_drive'] = vehicles['vehicle_left_hand_drive'].astype('category')
vehicles['journey_purpose_of_driver'] = vehicles['journey_purpose_of_driver'].astype('category')
vehicles['vehicle_location_restricted_lane'] = vehicles['vehicle_location_restricted_lane'].astype('category')

casualties['pedestrian_location'] = casualties['pedestrian_location'].astype('category')
casualties['pedestrian_movement'] = casualties['pedestrian_movement'].astype('category')
casualties['car_passenger'] = casualties['car_passenger'].astype('category')
casualties['bus_or_coach_passenger'] = casualties['bus_or_coach_passenger'].astype('category')
casualties['pedestrian_road_maintenance_worker'] = casualties['pedestrian_road_maintenance_worker'].astype('category')

```

6.4 Final Verification

After completing the data cleaning process, a final verification step was performed using dataset summaries. This confirmed that:

- Data types were correctly assigned
- No unintended data inconsistencies remained
- The datasets were clean and ready for further analysis

```
casualties.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 640522 entries, 0 to 640521
Data columns (total 20 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   collision_index                       640522 non-null object
 1   collision_year                       640522 non-null int64
 2   collision_ref_no                     640522 non-null object
 3   vehicle_reference                   640522 non-null int64
 4   casualty_reference                  640522 non-null int64
 5   casualty_class                      640522 non-null category
 6   sex_of_casualty                    640522 non-null category
 7   age_of_casualty                    640522 non-null float64
 8   casualty_severity                   640522 non-null int64
 9   pedestrian_location                 640522 non-null category
10   pedestrian_movement                 640522 non-null category
11   car_passenger                      640522 non-null category
12   bus_or_coach_passenger              640522 non-null category
13   pedestrian_road_maintenance_worker  640522 non-null category
14   casualty_type                      640522 non-null category
15   casualty_imd_decile                 571039 non-null float64
16   lsoa_of_casualty                   640522 non-null object
17   casualty_injury_based               640522 non-null int64
18   casualty_adjusted_severity_serious  640522 non-null float64
19   casualty_adjusted_severity_slight  640522 non-null float64
dtypes: category(8), float64(4), int64(5), object(3)
memory usage: 63.5+ MB

```

vehicles.info()

```
*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 920692 entries, 0 to 920691
Data columns (total 27 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   collision_index                           920692 non-null object
1   collision_year                           920692 non-null int64
2   collision_ref_no                         920692 non-null object
3   vehicle_reference                       920692 non-null int64
4   vehicle_type                             920692 non-null category
5   towing_and_articulation                 920692 non-null category
6   vehicle_manoeuvre                      920692 non-null category
7   vehicle_direction_from                 920692 non-null category
8   vehicle_direction_to                   920692 non-null category
9   vehicle_location_restricted_lane       920692 non-null category
10  junction_location                      920692 non-null category
11  skidding_and_overtaking                920692 non-null category
12  hit_object_in_carriageway              920692 non-null category
13  vehicle_leaving_carriageway            920692 non-null category
14  hit_object_off_carriageway             920692 non-null category
15  first_point_of_impact                   920692 non-null category
16  vehicle_left_hand_drive                 920692 non-null category
17  journey_purpose_of_driver                 920692 non-null category
18  sex_of_driver                          920692 non-null float64
19  age_of_driver                          920692 non-null float64
20  engine_capacity_cc                     920692 non-null float64
21  propulsion_code                        698492 non-null float64
22  age_of_vehicle                         698259 non-null float64
23  generic_make_model                     920692 non-null object
24  driver_imd_decile                      731307 non-null float64
25  lsoa_of_driver                         920692 non-null object
26  escooter_flag                          920692 non-null int64

dtypes: category(14), float64(6), int64(3), object(4)
memory usage: 103.6+ MB
```

collisions.info()

```
*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 503475 entries, 0 to 503474
Data columns (total 40 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   collision_index                           503475 non-null object
1   collision_year                           503475 non-null int64
2   collision_ref_no                         503475 non-null object
3   location_easting_osgr                   503475 non-null float64
4   location_northing_osgr                 503475 non-null float64
5   longitude                               503475 non-null float64
6   latitude                                503475 non-null float64
7   police_force                           503475 non-null int64
8   collision_severity                      503475 non-null int64
9   number_of_vehicles                      503475 non-null int64
10  number_of_casualties                    503475 non-null int64
11  date                                    503475 non-null datetime64[ns]
12  day_of_week                             503475 non-null int64
13  time                                    503475 non-null datetime64[ns]
14  local_authority_district                90402 non-null float64
15  local_authority_ons_district            503475 non-null object
16  local_authority_highway                 503475 non-null object
17  local_authority_highway_current         503475 non-null object
18  first_road_class                        503475 non-null int64
19  first_road_number                       503475 non-null int64
20  road_type                              503475 non-null category
21  speed_limit                            503475 non-null float64
22  junction_detail                         503475 non-null category
23  junction_control                        291767 non-null category
24  second_road_class                       503475 non-null float64
25  second_road_number                     312437 non-null float64
26  pedestrian_crossing                    503475 non-null category
27  light_conditions                       503475 non-null category
28  weather_conditions                     503475 non-null category
29  road_surface_conditions                 503475 non-null category
30  special_conditions_at_site              439745 non-null category
31  carriageway_hazards                    503475 non-null category
32  urban_or_rural_area                    503475 non-null category
33  did_police_officer_attend_scene_of_accident 503475 non-null category
34  trunk_road_flag                        467555 non-null float64
35  lsoa_of_accident_location              503475 non-null object
36  enhanced_severity_collision             262156 non-null float64
37  collision_injury_based                  503475 non-null int64
38  collision_adjusted_severity_serious      503475 non-null float64
39  collision_adjusted_severity_slight      503475 non-null float64

dtypes: category(11), datetime64[ns](2), float64(12), int64(9), object(6)
memory usage: 116.7+ MB
```

7. Feature Transformation

Feature transformation was performed to enhance the dataset by deriving meaningful variables from existing attributes. This step improves the representational quality of the data and prepares it for further analytical and modeling processes.

```
# Extract temporal features
collisions['hour'] = collisions['time'].dt.hour
collisions['month'] = collisions['date'].dt.month
collisions['year'] = collisions['date'].dt.year

# Verify extracted features
collisions[['hour', 'month', 'year']].head()
```

	hour	month	year
0	22	5	2021
1	15	10	2021
2	18	12	2020
3	16	12	2021
4	9	4	2021

7.1 Temporal Feature Extraction

Time-based features were extracted from the existing date and time variables to capture temporal patterns in road traffic collisions.

The following features were created:

- **hour**: extracted from the time variable to identify peak accident periods during the day
- **month**: extracted from the date variable to capture seasonal variations
- **year**: extracted to preserve temporal trends across multiple years

These features enable the analysis of temporal trends and, These features enable the model to capture temporal patterns such as peak traffic hours, seasonal variations, and long-term trends, which are important factors influencing collision severity.

7.2 Derived Features

Key variables already present in the dataset were retained to represent accident characteristics, including:

- **number_of_vehicles** — indicating the scale of the collision
- **number_of_casualties** — representing the impact severity

These variables provide important contextual information about accident intensity and contribute significantly to modeling collision severity.

8. Encoding

Categorical variables in the dataset were transformed into numerical representations to enable their use in machine learning models.

8.1 One-Hot Encoding

One-hot encoding was applied to selected categorical variables using the `pd.get_dummies()` function. Due to the large size of the dataset and memory limitations, encoding was restricted to the most relevant categorical features, including:

- weather_conditions
- light_conditions
- road_type
- urban_or_rural_area
- junction_control

This selective approach prevents excessive growth in dimensionality while retaining the most informative variables for modeling.

The dataset was first divided into **training and testing sets**, and one-hot encoding was applied separately to each subset. To ensure consistency between both sets, column alignment was performed to ensure that both training and testing datasets have identical feature spaces, preventing mismatches during model training and evaluation. This approach also prevents data leakage by ensuring that transformations are learned only from the training data.

```
[66]
✓ 3s # Step 1: Separate features and target
X = collisions.drop('collision_severity', axis=1)
y = collisions['collision_severity']

# Step 2: Split FIRST (IMPORTANT)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 3: Select ONLY important categorical columns
categorical_cols = [
    'weather_conditions',
    'light_conditions',
    'road_type',
    'urban_or_rural_area',
    'junction_control'
]

# Step 4: Apply One-Hot Encoding separately
X_train = pd.get_dummies(X_train, columns=categorical_cols, drop_first=True)
X_test = pd.get_dummies(X_test, columns=categorical_cols, drop_first=True)

# Step 5: Align columns (VERY IMPORTANT)
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)

# Step 6: Check result
print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)

*** Train shape: (402780, 60)
Test shape: (100695, 60)
```

The resulting encoded datasets maintained consistent dimensions, as shown by identical feature counts in both training and testing sets.

8.2 Justification for Method Used

One-hot encoding was chosen because the selected categorical variables are nominal and do not have an inherent order. This method avoids introducing misleading ordinal relationships that could negatively affect model performance.

Additionally, selective encoding was implemented to balance model performance with computational efficiency. Encoding all categorical variables would significantly increase the number of features and lead to memory constraints. By focusing on key variables, the dataset remains manageable while preserving important information.

9. Scaling / Normalization

Scaling was applied to numerical features to ensure that all variables contribute equally to the model and to improve model performance.

9.1 Standardization

Standardization was performed using the **StandardScaler** technique.

To maintain a proper machine learning pipeline and avoid data leakage, the dataset was first split into **training and testing sets**. The scaler was then fitted only on the training data and applied to both training and testing datasets.

Only numerical features with more than two unique values were selected for scaling. Binary variables (resulting from one-hot encoding) were excluded, as they are already in a normalized form.

The transformation ensures that the selected features have a mean close to zero and a standard deviation close to one, as confirmed by the statistical summary. Standardization is particularly important for distance-based and gradient-based algorithms, where feature scaling directly impacts model performance.

```
] 2s ▶ from sklearn.preprocessing import StandardScaler

# Identify numerical columns
numerical_cols = X_train.select_dtypes(include=['int64', 'float64']).columns

# Remove binary columns (0/1)
numerical_cols = [col for col in numerical_cols if X_train[col].nunique() > 2]

# Apply scaling
scaler = StandardScaler()

X_train[numerical_cols] = scaler.fit_transform(X_train[numerical_cols])
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])
print(X_train[numerical_cols].describe().head())
```

```

***
count    collision_year    location_easting_osgr    location_northing_osgr    \
mean     -4.363492e-17     -7.938434e-18           -1.106088e-17
std      1.000001e+00      1.000001e+00           1.000001e+00
min      -1.470783e+00      -4.167995e+00           -1.809545e+00
25%      -7.513779e-01      -6.741038e-01           -6.840437e-01

count    longitude    latitude    police_force    number_of_vehicles    \
mean     1.063750e-17    -2.496196e-18           2.014598e-17           3.340316e-17
std      1.000001e+00    1.000001e+00           1.000001e+00           1.000001e+00
min      -4.627833e+00    -1.858877e+00           -1.090976e+00           -1.208347e+00
25%      -6.657342e-01    -6.843643e-01           -9.675623e-01           -1.208347e+00

count    number_of_casualties    day_of_week    local_authority_district    \
mean     1.468610e-17           -5.821518e-19           5.204434e-18
std      1.000001e+00           1.000001e+00           1.000007e+00
min      -3.924350e-01           -1.621241e+00           -1.243638e+00
25%      -3.924350e-01           -1.104355e+00           -9.664293e-01

count    first_road_class    first_road_number    speed_limit    second_road_class    \
mean     -2.674370e-17         -1.555933e-17         -6.231670e-17           1.315354e-17
std      1.000001e+00         1.000001e+00         1.000001e+00           1.000001e+00
min      -2.205958e+00         -4.982749e-01         -1.124132e+00           -1.254936e+00
25%      -8.365942e-01         -4.982749e-01         -4.212222e-01           -1.254936e+00

count    second_road_number    enhanced_severity_collision    \
mean     1.409852e-17           -1.765745e-17
std      1.000002e+00           1.000002e+00
min      -3.125952e-01           -1.787873e+00
25%      -3.125952e-01           -5.307459e-01

count    collision_adjusted_severity_serious    collision_adjusted_severity_slight
mean     2.267305e-17                           1.210170e-17
std      1.000001e+00                           1.000001e+00
min      -5.830817e-01                           -1.801986e+00
25%      -5.830817e-01                           1.504022e-01

```

The statistical summary confirms successful standardization, with feature means centered around zero and standard deviations close to one.

9.2 Justification for Method Used

Standardization was chosen because the dataset contains numerical variables with different scales and ranges. Applying StandardScaler ensures that these features are comparable and prevents features with larger numerical scales from dominating the learning process, ensuring balanced contribution across all variables.

Additionally, applying scaling after splitting the dataset ensures that no information from the test data influences the training process, thereby preserving the integrity and reliability of model evaluation.

10. Univariate Analysis

Univariate analysis was conducted to examine the distribution and characteristics of individual variables in the dataset. This step provides a foundational understanding of the data by analyzing each feature independently.

10.1 Target Variable Distribution

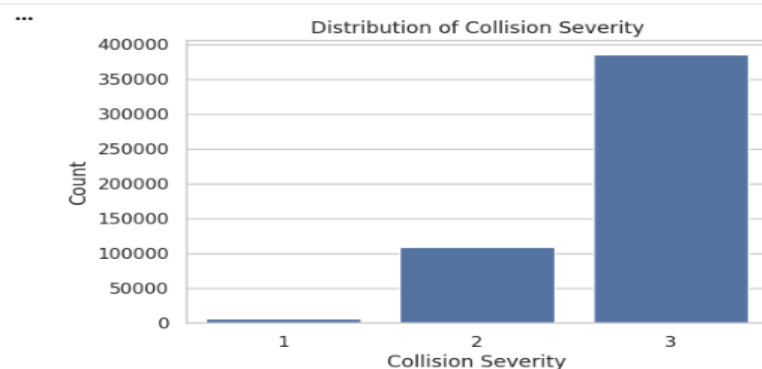
The distribution of the target variable, **collision_severity**, was analyzed using a count plot.

The analysis reveals that the dataset is **imbalanced**, with the majority of observations belonging to severity level 3. This indicates that less severe collisions are more frequent compared to more severe cases.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set style
sns.set(style="whitegrid")

# 1. Target Variable Distribution
plt.figure(figsize=(6,4))
sns.countplot(x='collision_severity', data=collisions)
plt.title("Distribution of Collision Severity")
plt.xlabel("Collision Severity")
plt.ylabel("Count")
plt.show()
```



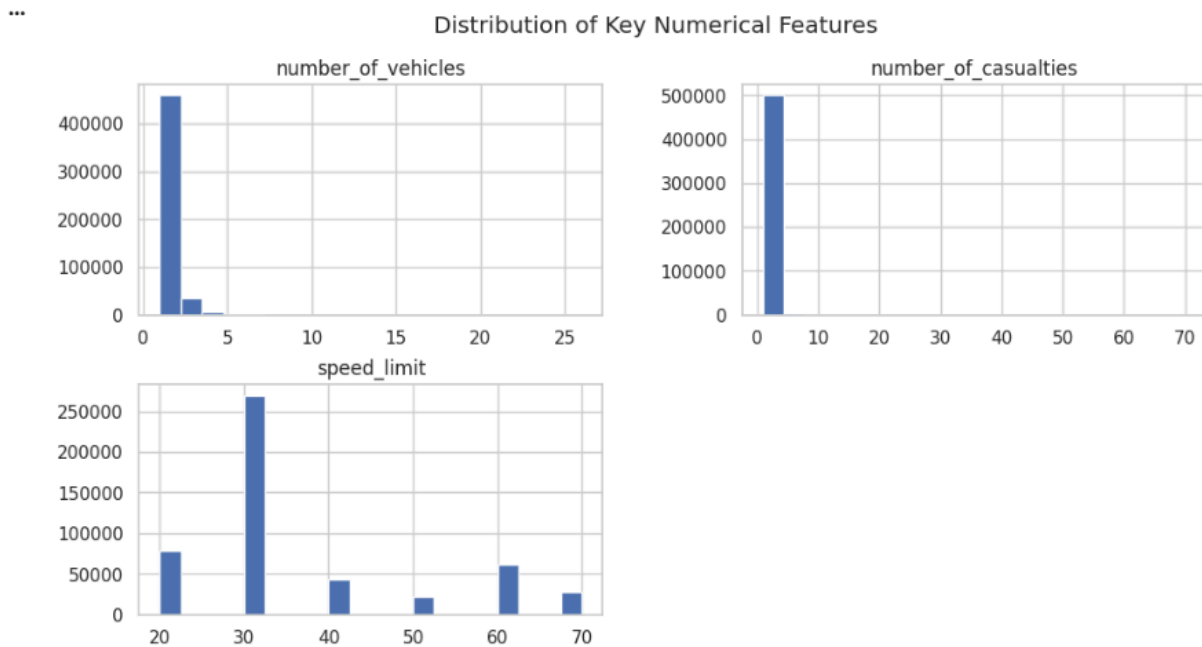
10.2 Distribution of Numerical Features

Key numerical variables, including **number_of_vehicles**, **number_of_casualties**, and **speed_limit**, were analyzed using histograms.

- The distribution of **number_of_vehicles** shows that most accidents involve a small number of vehicles, typically one or two.
- The **number_of_casualties** is heavily skewed toward lower values, indicating that most collisions result in few or no casualties.

- The **speed_limit** variable shows concentration around common limits such as 30 and 40, suggesting that a large proportion of accidents occur in urban areas.

```
# -----  
# 2. Numerical Features - Histogram  
# -----  
numerical_cols = ['number_of_vehicles', 'number_of_casualties', 'speed_limit']  
  
collisions[numerical_cols].hist(bins=20, figsize=(12,6))  
plt.suptitle("Distribution of Key Numerical Features")  
plt.show()
```



10.3 Outlier Detection

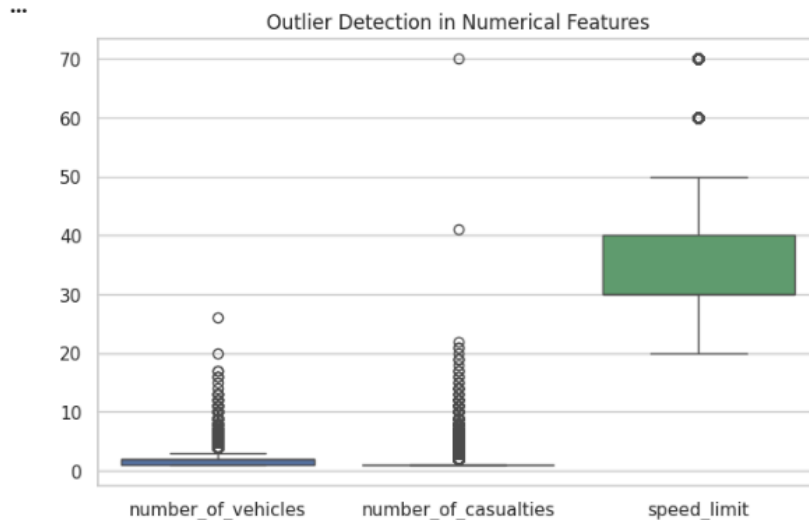
Boxplots were used to identify potential outliers in numerical features.

The analysis indicates the presence of **extreme values (outliers)**, particularly in the number of casualties and vehicles involved. These outliers represent rare but severe accident cases and are important for understanding the variability in the dataset.

```

# -----
# 3. Boxplot (Outlier Detection)
# -----
plt.figure(figsize=(8,5))
sns.boxplot(data=collisions[numerical_cols])
plt.title("Outlier Detection in Numerical Features")
plt.show()

```



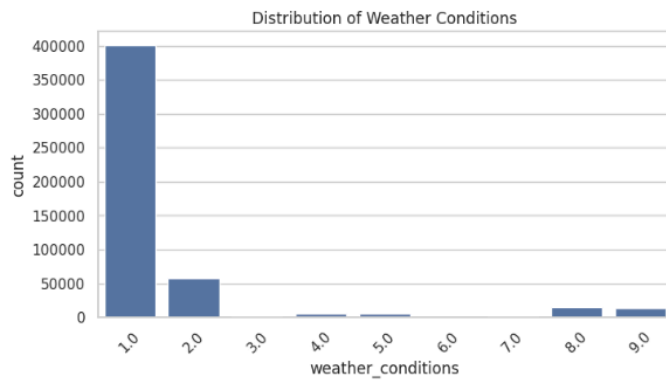
10.4 Distribution of Categorical Features

Categorical variables such as **weather_conditions**, **road_type**, **urban_or_rural_area**, and **light_conditions** were analyzed using count plots.

- The distribution of **weather_conditions** shows that most accidents occur under normal weather conditions, indicating that adverse weather is not the primary contributing factor in most cases.
- The **road_type** analysis suggests that certain road categories experience higher accident frequencies, reflecting differences in traffic density and road usage.
- The **urban_or_rural_area** variable indicates that a majority of accidents occur in **urban areas**, likely due to higher traffic volume.
- The distribution of **light_conditions** shows that most accidents occur during daylight, although a significant number also occur under low-light conditions.

1
2

```
# -----  
# 4. Categorical Features Distribution  
# -----  
  
# Weather Conditions  
plt.figure(figsize=(8,4))  
sns.countplot(x='weather_conditions', data=collisions)  
plt.title("Distribution of Weather Conditions")  
plt.xticks(rotation=45)  
plt.show()
```

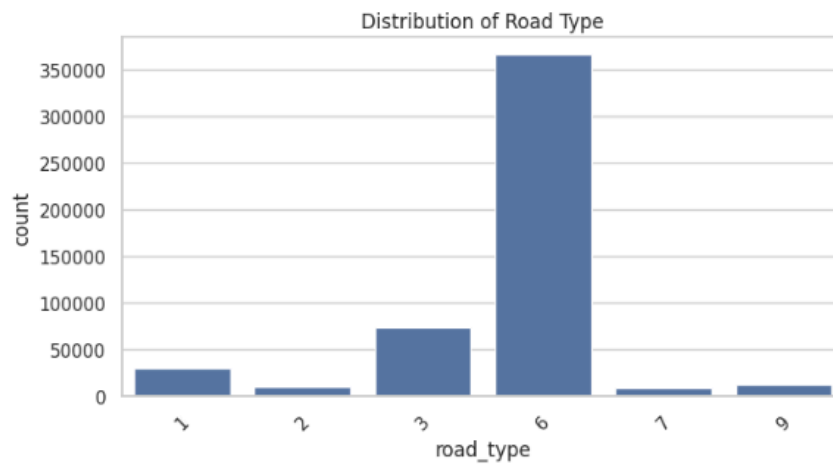


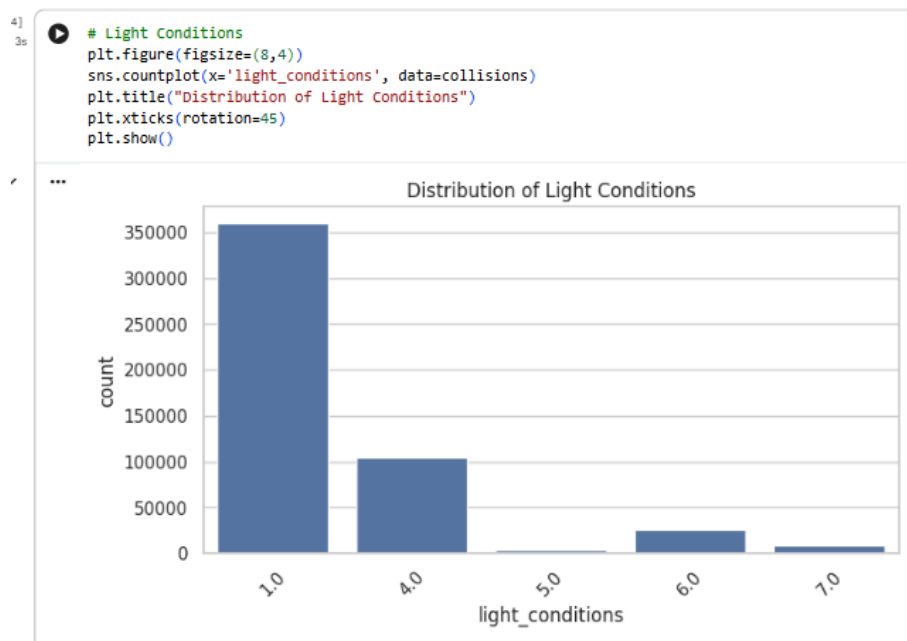
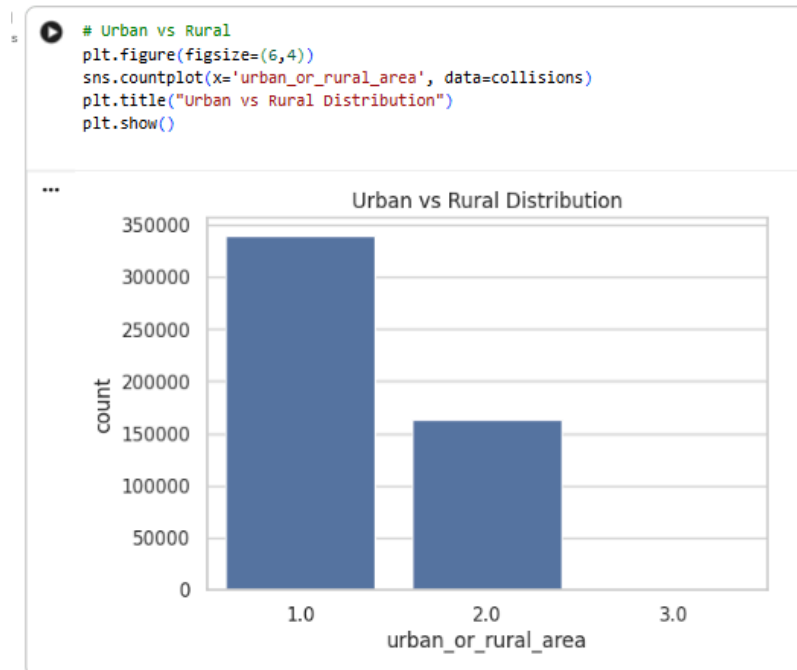
3



```
# Road Type  
plt.figure(figsize=(8,4))  
sns.countplot(x='road_type', data=collisions)  
plt.title("Distribution of Road Type")  
plt.xticks(rotation=45)  
plt.show()
```

...





11. Bivariate Analysis

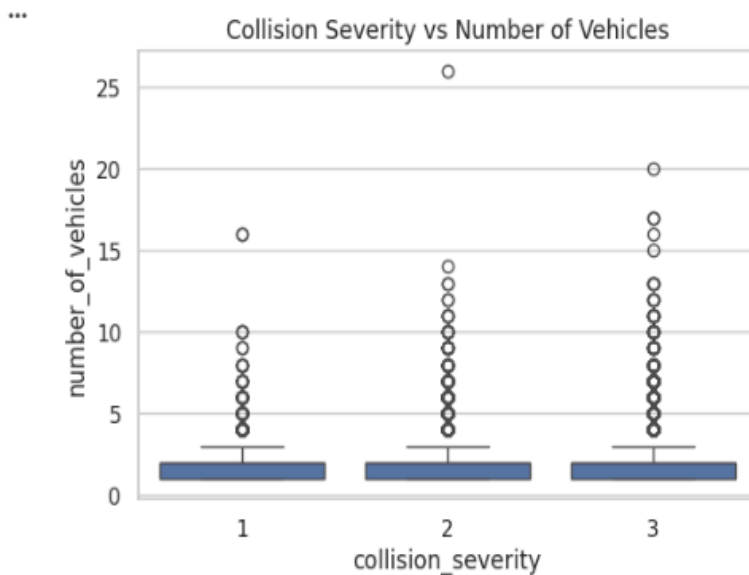
Bivariate analysis was performed to examine the relationships between pairs of variables, particularly focusing on how different features influence the target variable **collision_severity**.

11.1 Numerical Features vs Collision Severity

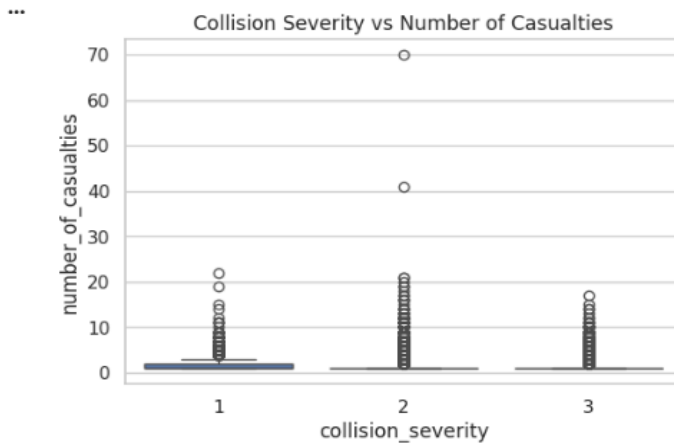
Boxplots were used to analyze the relationship between numerical variables and collision severity.

- The number of vehicles shows a slight increase with higher severity levels, indicating that multi-vehicle collisions may lead to more severe outcomes.
- The number of casualties increases significantly with severity, confirming a strong relationship between accident severity and the number of people affected.

```
#step11)Collision Severity vs Numerical Features (Boxplot)
# Severity vs Number of Vehicles
plt.figure(figsize=(6,4))
sns.boxplot(x='collision_severity', y='number_of_vehicles', data=collisions)
plt.title("Collision Severity vs Number of Vehicles")
plt.show()
```



```
# Severity vs Number of Casualties
plt.figure(figsize=(6,4))
sns.boxplot(x='collision_severity', y='number_of_casualties', data=collisions)
plt.title("Collision Severity vs Number of Casualties")
plt.show()
```



11.2 Categorical Features vs Collision Severity

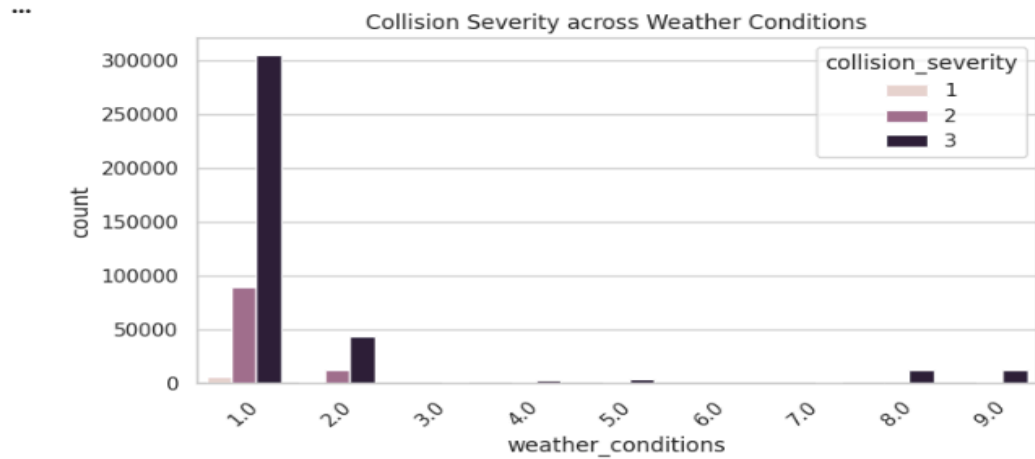
Count plots with hue were used to compare categorical variables against collision severity.

- The distribution across **weather conditions** shows that most accidents occur under normal conditions; however, severity levels vary across categories.
- The **road type** analysis indicates that certain road types experience higher severity levels, possibly due to traffic conditions and road design.
- **Light conditions** reveal that while many accidents occur during daylight, severe accidents are also present in low-light conditions.
- The **urban vs rural** comparison suggests that although most accidents occur in urban areas, rural accidents may show higher severity in certain cases.

```

# Collision Severity vs Categorical Features (Countplot)
# Weather vs Severity
plt.figure(figsize=(8,4))
sns.countplot(x='weather_conditions', hue='collision_severity', data=collisions)
plt.title("Collision Severity across Weather Conditions")
plt.xticks(rotation=45)
plt.show()

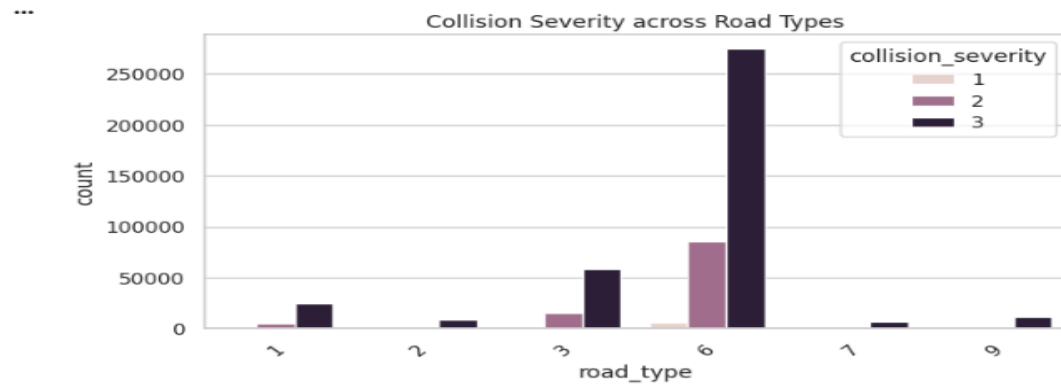
```



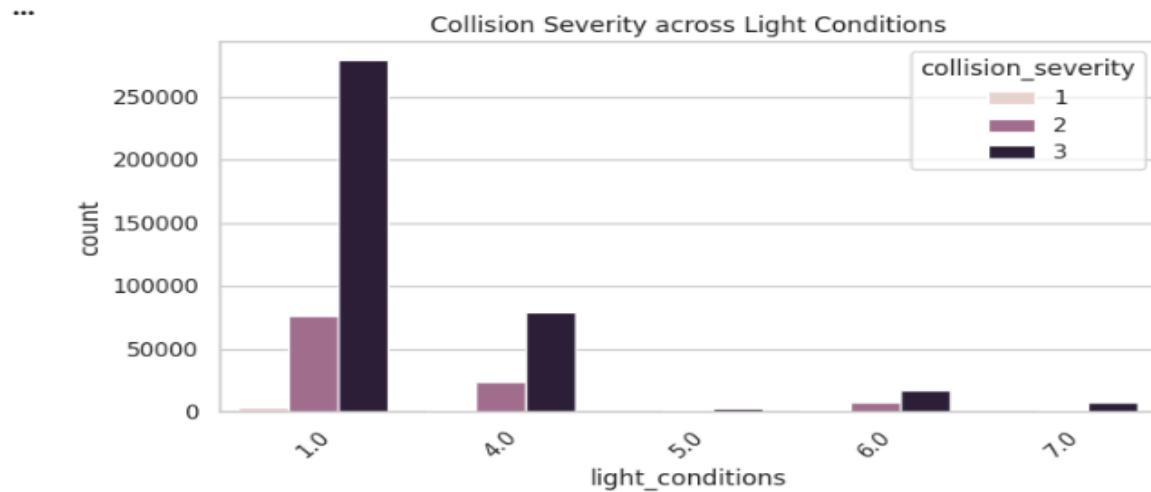
```

# Road Type vs Severity
plt.figure(figsize=(8,4))
sns.countplot(x='road_type', hue='collision_severity', data=collisions)
plt.title("Collision Severity across Road Types")
plt.xticks(rotation=45)
plt.show()

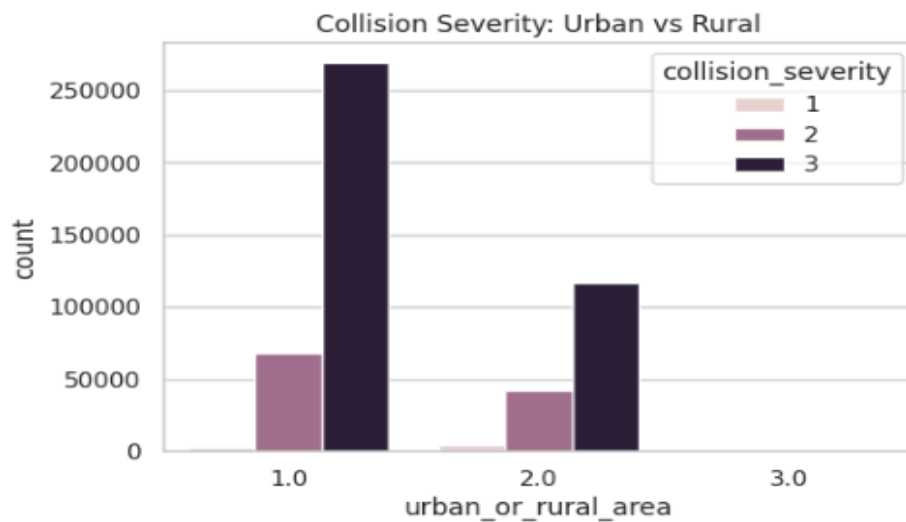
```



```
# Light Conditions vs Severity
plt.figure(figsize=(8,4))
sns.countplot(x='light_conditions', hue='collision_severity', data=collisions)
plt.title("Collision Severity across Light Conditions")
plt.xticks(rotation=45)
plt.show()
```



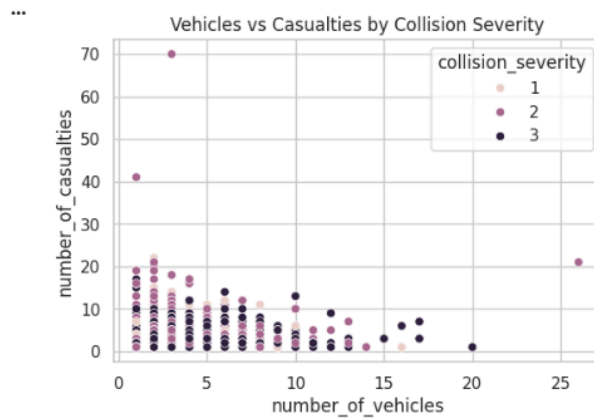
```
# Urban vs Rural
plt.figure(figsize=(6,4))
sns.countplot(x='urban_or_rural_area', hue='collision_severity', data=collisions)
plt.title("Collision Severity: Urban vs Rural")
plt.show()
```



11.3 Relationship Between Numerical Variables

A scatter plot between **number_of_vehicles** and **number_of_casualties** shows a positive relationship, where an increase in the number of vehicles involved often corresponds to a higher number of casualties.

```
#Scatter Plot (Numerical Relationship)
plt.figure(figsize=(6,4))
sns.scatterplot(x='number_of_vehicles', y='number_of_casualties', hue='collision_severity', data=collisions)
plt.title("Vehicles vs Casualties by Collision Severity")
plt.show()
```



11.4 Regression Analysis

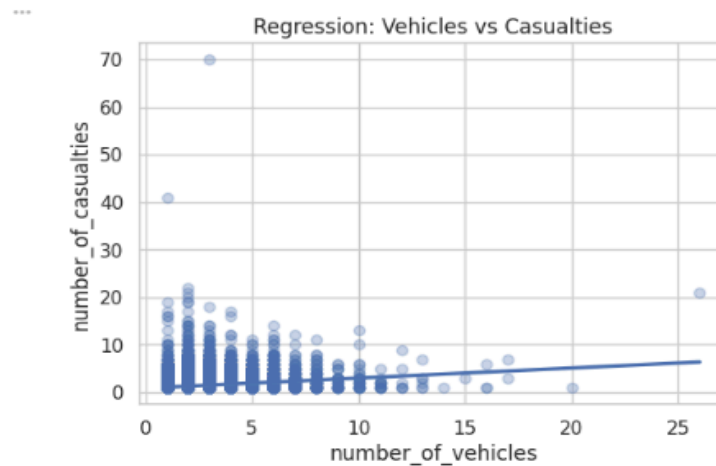
A regression plot was used to examine the trend between the number of vehicles and number of casualties.

A slight positive trend is observed, indicating that accidents involving more vehicles tend to result in higher casualties. However, the wide dispersion suggests that this relationship alone is not strongly predictive.

```

# Regression Plot (Trend between Vehicles and Casualties)
# Confirms slight positive trend but large spread reduces predictive strength
plt.figure(figsize=(6,4))
sns.regplot(
    x='number_of_vehicles',
    y='number_of_casualties',
    data=collisions,
    scatter_kws={'alpha':0.3}
)
plt.title("Regression: Vehicles vs Casualties")
plt.show()

```



11.5 Distribution Insight (Casualties)

The distribution of casualties is highly right-skewed, with most accidents resulting in very few casualties. Extreme values are rare but indicate severe accident scenarios.

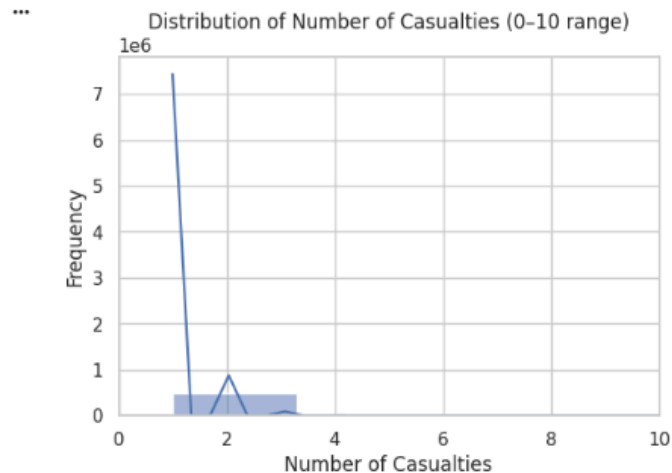
```

# Histogram (Casualties Distribution - Right Skewed)
# Most accidents involve 0-2 casualties; extreme values are rare but present
plt.figure(figsize=(6,4))

sns.histplot(
    collisions['number_of_casualties'],
    kde=True,
    bins=30
)

plt.xlim(0, 10) # focus on realistic range
plt.title("Distribution of Number of Casualties (0-10 range)")
plt.xlabel("Number of Casualties")
plt.ylabel("Frequency")
plt.show()

```



11.6 Multivariate Analysis (Pairplot)

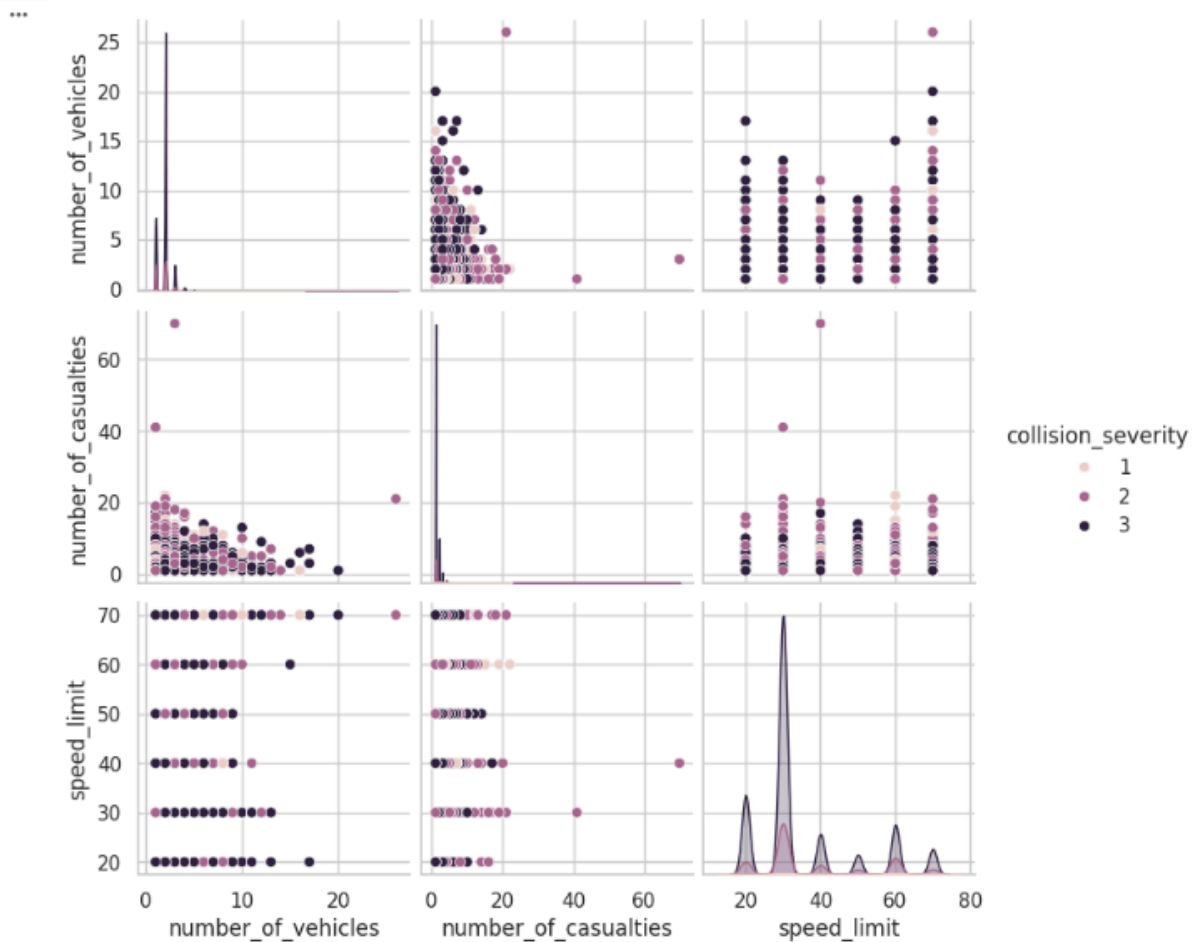
A pairplot was used to analyze relationships among key numerical variables. Most feature interactions show weak correlations, indicating that variables contribute independently with limited multicollinearity.

```

# Pairplot (Multivariate Relationships)
# Most features show weak correlations → low multicollinearity
important_cols = [
    'collision_severity',
    'number_of_vehicles',
    'number_of_casualties',
    'speed_limit'
]

sns.pairplot(collisions[important_cols], hue='collision_severity')
plt.show()

```



11.7 Aggregated Analysis (Grouped Trends)

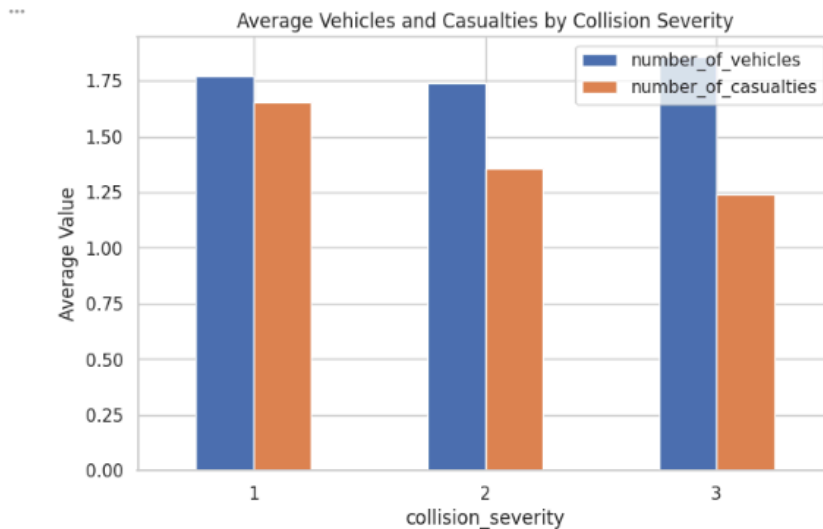
Aggregated analysis shows that the average number of casualties increases with collision severity, while the number of vehicles shows relatively smaller variation across severity levels.

```

# Grouped Analysis (Average Values by Severity)
# Casualties increase with severity; vehicles show minor variation
grouped = collisions.groupby('collision_severity')[
    ['number_of_vehicles', 'number_of_casualties']
].mean()

grouped.plot(kind='bar', figsize=(8,5))
plt.title("Average Vehicles and Casualties by Collision Severity")
plt.ylabel("Average Value")
plt.xticks(rotation=0)
plt.show()

```

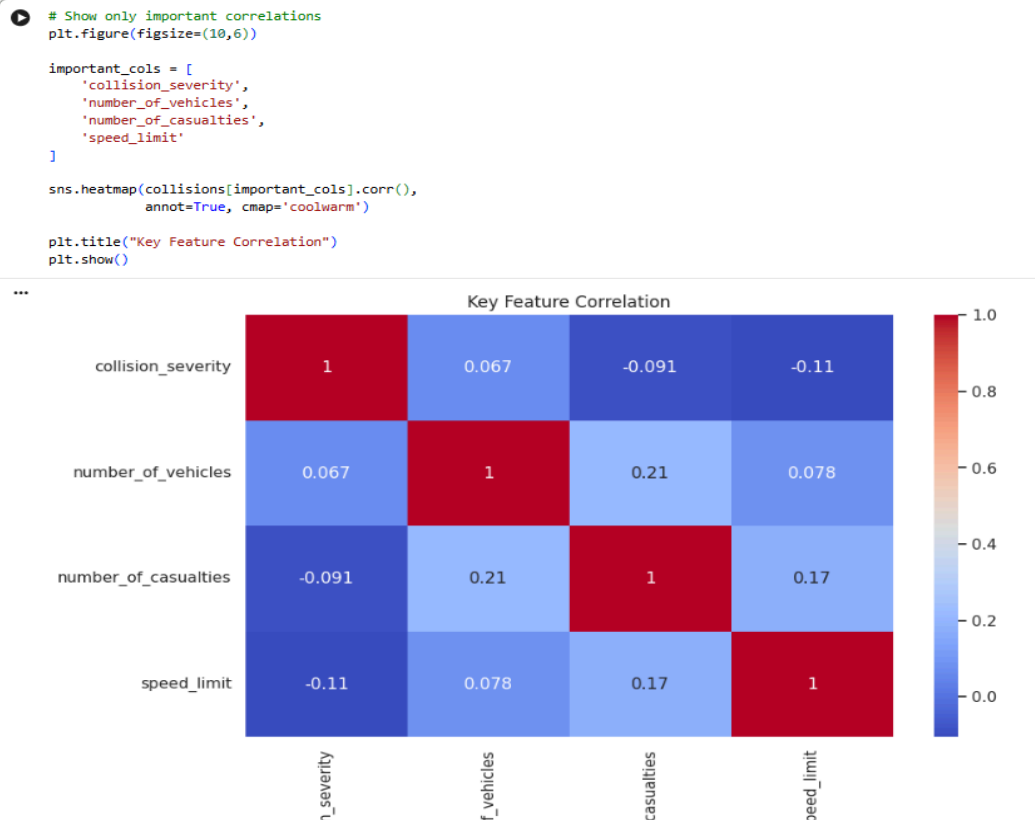


11.7 Correlation Analysis

A heatmap was used to visualize correlations among key numerical variables.

The analysis shows that **number_of_casualties** has a stronger positive relationship with **collision_severity**, indicating that more severe accidents involve higher casualties. The **number_of_vehicles** shows a weaker positive relationship with severity, while **speed_limit** has a low correlation.

Overall, casualty-related variables have the most significant influence on collision severity, while other features show limited direct relationships.



12. Visualizations with Insights

The EDA reveals clear patterns in accident severity and contributing factors.

- The dataset is **highly imbalanced**, with most accidents belonging to lower severity levels. This may bias model predictions toward majority classes if not handled properly.
- **number_of_casualties** shows the strongest relationship with collision severity, indicating that severity is primarily driven by the impact of the accident rather than the number of vehicles alone.
- **number_of_vehicles** has only a weak positive relationship with severity, suggesting that multi-vehicle accidents are not necessarily more severe in all cases.
- The distribution of numerical features is **heavily skewed**, with most observations concentrated at low values and a small number of extreme cases representing severe accidents.

- Categorical features such as **weather_conditions** and **light_conditions** show that most accidents occur under normal conditions, indicating that severity is influenced more by accident dynamics than environmental factors.
- Correlation analysis confirms that most features are **weakly correlated**, meaning they contribute independently and reduce the risk of multicollinearity in the model.

13. Conclusion

This study analyzed road traffic collision data to understand the factors influencing accident severity and to support predictive modeling. The analysis revealed that collision severity is highly imbalanced, with most cases classified as slight, which has important implications for model development and evaluation.

Key findings indicate that the **number of casualties** is the most influential factor associated with higher severity levels, while the **number of vehicles** shows only a weak relationship, suggesting that severity is more dependent on impact outcomes than collision scale. Environmental factors such as **light conditions**, **weather conditions**, and **road type** also contribute to variations in accident severity, reflecting the role of visibility, infrastructure, and driving conditions.

The exploratory analysis further showed that most numerical features are highly skewed, with a small number of extreme cases representing severe accidents. Correlation analysis indicates that features are generally weakly correlated, meaning they contribute independently to the prediction task without significant multicollinearity.

Overall, the dataset provides meaningful insights into accident patterns and supports the development of machine learning models for severity prediction. These findings can help in improving road safety strategies, such as enhancing lighting conditions, managing high-risk road types, and focusing on factors that contribute to severe casualties.

Future work may include implementing and optimizing predictive models and deploying them in real-time systems for accident risk assessment and prevention.

